

## 📖 OOAD Ch12: UML 類別圖與物件關係

**主題重點：** [UML Class Diagram](#) 描述的是系統中類別與物件之間的靜態結構。本章整理類別圖基本語法、六大物件關係、進階類別圖概念，以及老師特別提醒的「分類 (Classification)」與「一般化 (Generalization)」差異。

### 📊 類別圖基本概念

#### 1. 類別圖描述靜態關係

[Class Diagram](#) 描述的是系統中物件的 **靜態關聯 (Static Relationship)**。

也就是說，它關心的是：

- 系統中有哪些物體型別 (type) 或類別 (class)
- 類別之間有什麼結構關係
- 哪些類別知道、擁有、使用或繼承其他類別

它不是用來描述：

- 程式實際執行流程
- 方法呼叫順序
- 動態互動細節

如果要描述動態行為，通常會使用其他 UML 圖，例如 [Sequence Diagram](#) 或 [Activity Diagram](#)。

#### 2. 類別的基本表示法

一個類別在 UML 中通常以矩形表示，分成三層：

```
Class Name
-----
Attributes / Variables
-----
Operations / Methods
```

三層意義如下：

1. **Class Name** (類別名稱)
  - 類別的名稱。
2. **Attributes / Variables** (屬性 / 變數)
  - 儲存物件的狀態。
3. **Operations / Functions / Methods** (操作 / 方法)
  - 類別所擁有的功能。

UML 的標準 operation 語法為：

```
visibility name(parameter list): return-type-expression {property string}
```

例如：

```
+ computeSalary(): Money
- updateCache(key: String): void
```

#### 3. 可見性 (Visibility)

可見性用來規範屬性與方法的存取權限。

| 符號 | 意義        | 說明               |
|----|-----------|------------------|
| +  | Public    | 系統內任何物件都可以存取     |
| -  | Private   | 只有定義該成員的類別本身可以存取 |
| #  | Protected | 類別本身與子類別可以存取     |

補充：

- 在 [Java](#) 中，[protected](#) 除了子類別外，同一個 package 內的其他類別也可以存取。
- 在 [C++](#) 中，[friend](#) 類別或函式可以完全存取另一個類別的所有成員。

### 🔗 物件之間的六大關係

物件導向程式可以被視為一組互相互動的物件集合。UML 用不同的圖形符號來表達這些物件與類別之間的關係。

本章主要整理六種關係：

1. [Dependency](#)
2. [Association](#)
3. [Generalization / Inheritance](#)
4. [Realization / Implementation](#)
5. [Aggregation](#)

## 1 相依關係 (Dependency) : Uses a

### 定義

Dependency 表示:

當類別 X 使用到類別 Y 時, X 就相依於 Y。

這通常是一種短暫的使用關係, 不一定代表 X 長期保存 Y。

### UML 圖示

X - - - - -> Y

- 使用虛線箭頭。
- 箭頭從使用者 X 指向被使用者 Y。

### 程式碼實作情境

只要 X 類別內部使用到 Y, 就可能構成 dependency, 例如:

```
void f1(Y y);  
void f2(Y* y);
```

C++

或是在方法內宣告區域變數:

```
void X::foo() {  
    Y y;  
    y.Foo();  
}
```

C++

重點是:

- Y 不是 X 的長期成員變數。
- X 只是暫時使用 Y。

### 老師例子: 車子與行李

Car 有一個計算行李重量的功能:

```
void addBaggageWeight(Baggage* baggage)
```

C++

此時:

- Car 使用 Baggage。
- Car 不一定長期擁有 Baggage。
- 因此 Car dependency Baggage。

## 2 關聯關係 (Association) : Knows a

### 定義

Association 是一種長期、靜態的關係, 表示:

X 認識 Y, 或 X 知道 Y 的存在。

### UML 圖示

X ----- Y

- 使用實線。
- 可以有箭頭, 也可以沒有箭頭。

### 程式碼實作

如果 X 類別的 attribute 裡面有 Y, 通常就是 association:

```
class X {  
private:  
    Y* y;  
};
```

C++

也可以是 reference 或成員物件:

```
class X {  
private:  
    Y% y;  
};
```

C++

老師的重點是:

只要屬性裡面有 Y, 就是關聯關係。

Association 的重要屬性

1. 多重性 (Multiplicity)

Multiplicity 描述數量關係，例如：

| 標記   | 意義        |
|------|-----------|
| 1    | 一個        |
| *    | 多個        |
| 0..1 | 可有可無，最多一個 |
| 3..* | 三個以上      |

例子：

```
Customer 1 ----- * Order
```

代表一個 customer 可以有 multiple 筆 order。

2. 角色名稱 (Role Name)

Role name 寫在連線兩端，用來說明物件在關聯中扮演的角色。

例如 Employee 在某段關聯中標示為：

```
sale rep
```

代表該員工在這個關聯裡扮演業務代表的角色。

3. 導向性 (Navigability)

箭頭代表在實作中能從某一端導航到另一端。

例如：

```
Order ----> Customer
```

通常代表 Order 內部有一個指向 Customer 的 reference 或 pointer。

不過老師提醒：

- 在 UML 2.x 以後，已經移除強制使用箭頭的規定。
- 是否畫 navigability 要依照圖的目的與清晰度決定。

3 一般化 / 繼承 (Generalization / Inheritance) : Is a

定義

Generalization 對應到物件導向程式中的 Inheritance。

UML 之所以稱為 generalization，而不是 inheritance，是因為 UML 不想被特定程式語言綁死。

UML 圖示

```
Child -----▷ Parent
```

- 實線加空心三角形。
- 空心三角形指向父類別 (base class / superclass)。

多型設計重點

老師特別提醒：

為了保持 Polymorphism 的彈性，關聯線應該盡量指向父類別，而不是特定子類別。

例如：

```
Order ----- Customer
              /   \
CorporateCustomer  PersonalCustomer
```

Order 應該關聯到 Customer 父類別，而不是直接連到：

- CorporateCustomer
- PersonalCustomer

這樣未來新增新的 customer subtype 時，Order 不需要修改，才能活用多型。

4 實作關係 (Implementation / Realization) : Can do

定義

Realization 表示某個類別實作某個 Interface。

也就是說，該類別保證能執行 interface 中定義的方法。

## UML 圖示

常見畫法有兩種：

```
Class - - -> Interface
```

或在介面名稱上方標示：

```
<<interface>>  
Window
```

## 程式語言對應

在 **Java** 中：

```
class MyWindow implements Window {  
    ...  
}
```

JAVA

在 **C++** 中沒有純粹的 interface 語法，通常用：

- 沒有變數
- 只有純虛擬函式 (pure virtual functions)

的抽象類別替代。

## 老師例子：跨平台 UI 設計

假設要開發一個文字編輯器，如果核心業務邏輯直接呼叫特定平台 GUI API，例如：

- Java GUI API
- Windows GUI API
- `toFront()` 之類的平台特定方法

未來要移植到：

- Linux / X11
- macOS
- 其他 GUI framework

就會非常痛苦。

比較好的設計是：

- 定義一個純粹的 `Window` interface。
- 讓核心業務邏輯只相依於 `Window` interface。
- 針對不同平台實作不同 class：
  - `WindowsWindow`
  - `X11Window`
  - `MacWindow`
- 核心程式碼不需要知道背後是哪個平台。

這就是「相依於抽象，不相依於具體實作」的設計精神。

## 5 聚合 (Aggregation)：Has a / Part-of

### 定義

**Aggregation** 是一種較強的 association，用來表達整體與部分的關係。

它代表：

部分聚在一起形成整體，但部分可以脫離整體獨立存在。

## UML 圖示

```
Whole ◇----- Part
```

- 空心菱形。
- 菱形畫在整體 (whole / 大的那一端)。

## 生命週期特性

Aggregation 的特徵是：

- 部分可以獨立存在。
- 部分生命週期不受整體控制。
- 整體被刪除時，部分不會跟著被刪除。

## 例子

### 貨車與貨物

貨車可以載貨物，但貨物不是貨車生命週期的一部分。

- 貨車消失，貨物不一定消失。
- 貨物可以轉移到另一台貨車。

多邊形與填色樣式

Polygon 可以使用某個 Style 作為填色樣式。

但：

- Polygon 被刪除後，Style 仍然可以存在。
- Style 可以給其他圖形使用。

Larman 的提醒

老師補充：Larman 的 OOAD 書建議盡量不要使用 aggregation。

原因是：

- aggregation 的定義常常很模糊。
- 很多情況用一般 association 就足夠。

6 合成 (Composition)：Strong Has a / Part-of

定義

Composition 是比 aggregation 更強的整體 / 部分關係。

它代表：

部分不能脫離整體獨立存在，部分生命週期由整體控制。

UML 圖示

Whole ◆----- Part

- 實心菱形。
- 菱形畫在整體（whole / 大的那一端）。

生命週期特性

Composition 的關鍵是：

- part 不能獨立於 whole 存在。
- whole 被刪除時，part 必須跟著死亡。
- 這通常對應到 cascade delete。

例子：Polygon 與 Point

Polygon 由多個 Point 組成。

如果這些 Point 是專屬於該 Polygon 的頂點，那麼：

- 刪除 Polygon 時，構成它的 Point 也應該被刪除。
- Point 的生命週期由 Polygon 控制。

程式碼實作概念

例如在 C++ 中：

```
class Polygon {
private:
    Point* points;

public:
    Polygon() {
        points = new Point[10];
    }

    ~Polygon() {
        delete[] points;
    }
};
```

C++

在 constructor 中建立 part，在 destructor 中刪除 part，代表 whole 負責 part 的生命週期。

替代畫法

有時也可以直接把「部分」類別畫在「整體」類別框框裡面，表示更強烈的合成意義。

📁 進階概念 1: 物件圖 (Object Diagram / Instance Diagram)

定義

Object Diagram 描述的是系統在某一個瞬間的物件狀態快照 (snapshot)。

它不是描述 class 之間的抽象關係，而是描述實際存在的 objects。

圖示特徵

物件名稱底下會有底線，例如：

Don:Person  
engineering:Organization

其中：

- `Don` 是 object name。
- `Person` 是 class name。
- 底線表示它是 object，而不是 class。

### Object Diagram 的限制

Object Diagram 中的線表示物件之間的 link / association instance。

它通常不會畫 inheritance 線，因為 inheritance 是 class-level 的結構，不是某個時間點 object 之間的 link。

### 老師例子：Composite Design Pattern

老師用組織薪水計算說明 [Composite Design Pattern](#)。

需求：

- `Organization` 底下可以包含其他 organization。
- `Organization` 也可以包含 `Person`。
- 要計算整個組織的總薪水。

設計方式：

```
graph TD
    Party --|> Person
    Party --|> Organization
```

其中：

- `Party` 是抽象父類別。
- `Person` 和 `Organization` 都繼承 `Party`。
- `Party` 定義 `computeSalary()`。

計算方式：

- 遇到 `Person`：直接回傳該人的薪水。
- 遇到 `Organization`：遞迴呼叫底下每個 child party 的 `computeSalary()` 並加總。

這是一個非常經典、也不容易被誤用的物件導向設計模式。

## 🔑 進階概念 2：Class Scope Operations and Attributes

在 UML 類別圖中，如果 attribute 或 operation 名稱底下有底線，代表它是：

**Class Scope**

也就是程式語言中的 `static` 成員。

意義：

- 它屬於整個 class。
- 不是某個單一 object 獨有。
- 所有 instances 共享同一份 class-scope attribute 或 operation。

對應：

- C++： `static`
- Java： `static`

## 🎨 進階概念 3：多重與動態分類

### Multiple Classification（多重分類）

[Multiple Classification](#) 表示一個物件可以同時擁有多重角色。

注意：

多重分類不是多重繼承。

UML 中可以在繼承線附近加上 discriminator（區別者），例如：

- `sex`
- `role`

並可用 `{complete}` 表示分類已完整，不能再擴充新的子類別。

這可以幫助讀者了解哪些組合是合法的。

### Dynamic Classification（動態分類）

[Dynamic Classification](#) 表示物件在執行期間可以改變它的型別或分類。

例如：

- 一個 `Engineer` 轉職成 `Manager`
- 一個人從學生身分轉變為校友

UML 圖上通常會用：

```
<<dynamic>>
```

並以虛線標示。

實作重點：不能真的靠繼承完成

老師特別強調：

實務上不能真的用繼承來寫 dynamic classification。

也就是說，不能天真地設計成：

```
Person
├─ Engineer
└─ Manager
```

然後期待一個 `Engineer` object 在執行期間變成 `Manager` object。

原因與 CH11 的電影分類案例相同：

一個 object 不能在生命週期內改變自己的 class。

正確方式通常是使用 [State Design Pattern](#)：

```
class Person {
private:
    JobType* jobType;
};
```

C++

讓 `Person` 持有一個代表職位狀態的物件，例如：

- `EngineerJobType`
- `ManagerJobType`

當職位改變時，換掉 `jobType` 指向的物件，而不是換掉 `Person` 本身的 class。

## ÷ 進階概念 4：衍生關聯與屬性（Derived Associations and Attributes）

定義

[Derived Attribute](#) 或 [Derived Association](#) 指的是：

某些屬性或關聯不需要實際儲存在資料庫或物件變數中，而是可以透過其他資料計算出來。

UML 圖示

在屬性名稱或關聯名稱前面加上斜線 `/`：

```
/age
/balance
/employer
```

例子 1：年齡 `/age`

不應該在系統中真的儲存一個 `age` 變數。

原因是：

- 年齡會隨時間變動。
- 如果真的存 `age`，每年都要寫程式幫所有人加一。
- 這很容易出錯，也沒有必要。

比較好的方式是：

```
/age = currentDate - birthday
```

也就是從 `birthday` 與目前系統時間推導出來。

例子 2：銀行餘額 `/balance`

銀行帳戶的總餘額也可以視為 derived attribute。

```
/balance = sum(entries)
```

也就是把所有存提款紀錄加總得到，而不是手動維護一個容易不一致的欄位。

## 🔑 進階概念 5：限定關聯（Qualified Association）

定義

[Qualified Association](#) 類似程式設計中的：

- associative array
- map
- dictionary

也就是透過某個 key 來限定關聯對象。

## UML 圖示

在關聯的一端加上一個小方塊，裡面寫入 key / index。

例如：

```
Order [product] ----- OrderLine
```

代表 `Order` 可以透過 `product` 這個 key 找到對應的 `OrderLine`。

## 意義

在概念上，它保證：

在一個 `Order` 中，相同 `Product` 只能對應到唯一一筆 `OrderLine`。

如果要買兩件相同 T-shirt，應該在同一筆 `OrderLine` 中把數量改成 2，而不是建立兩筆同產品的 `OrderLine`。

## 進階概念 6：關聯類別 (Association Class)

### 定義

當兩個類別之間存在 association，尤其是多對多關係時，這段關聯本身可能也需要儲存屬性或行為。

這時可以把這段關係實體化一個：

```
Association Class
```

## UML 圖示

用一條虛線將普通類別連接到關聯實線上。

### 例子：Person 與 Company 的聘僱關係

`Person` 與 `Company` 之間有聘僱關係。

但「聘僱」本身也有自己的資料，例如：

- 聘僱期間 `period: dateRange`
- 職稱
- 薪資
- 合約狀態

所以可以抽出：

```
Employment
```

作為 association class。

也可以改畫成一般類別放在兩者中間：

```
Person ----- Employment ----- Company
```

此時原本的直接關聯可以視為 derived association，例如 `/employer`。

## 進階概念 7：參數化類別 (Parameterized Class / Template)

### 定義

`Parameterized Class` 是帶有型別參數的類別。

對應到不同語言：

- C++: `Template`
- Java: `Generics`

## UML 圖示

在類別方塊右上角加上一個虛線小框，裡面填入樣板參數，例如：

```
T
```

### 使用方式

其他類別使用它時，可以畫一條 dependency 虛線指向它，並標註：

```
<<bind>> <Employee>
```

表示把泛型參數 `T` 綁定為 `Employee` 型別。

## 特別釐清：Classification vs Generalization

老師特別強調，人類自然語言很容易混淆：



- [Classification](#) (分類)
- [Generalization](#) (一般化 / 繼承)

但這兩者不是同一件事。

### Generalization 具有傳遞性

Generalization / inheritance 具有 transitive 特性。

例如：

```
牧羊犬 is a kind of 狗
狗 is a kind of 動物
```

因此可以推論：

```
牧羊犬 is a kind of 動物
```

這是合理的。

### Classification 不具備傳遞性

Classification 則不是這樣。

例如：

```
牧羊犬 is a kind of 狗
狗 is a Species
```

如果用自然語言硬推，會得到：

```
牧羊犬 is a Species
```

這就很荒謬。

因為：

- 狗可以是一個物種 (species) 。
- 牧羊犬只是品種或分類下的子類型。
- 不能因為自然語言都用了「是一種」就把它當成 inheritance。

### 設計結論

在設計物件導向繼承時，不要只靠直覺的：

```
is a kind of
```

而要更嚴格檢查：

- 這是否真的具有 generalization 的傳遞性？
- 子類別是否真的能完全替代父類別？
- 這是穩定的型別差異，還是會動態改變的分類 / 角色？
- 如果分類會改變，是否應該改用 [State Design Pattern](#)？

## ✔ 本章核心觀念總結

1. [Class Diagram](#) 描述靜態結構，不是描述程式執行流程。
2. 類別圖基本結構包含 class name、attributes、operations。
3. +、-、# 分別代表 public、private、protected。
4. [Dependency](#) 是短暫使用關係，通常發生在參數、區域變數或方法呼叫中。
5. [Association](#) 是長期認識關係，通常對應到 attribute、pointer 或 reference。
6. [Generalization](#) / [Inheritance](#) 是「is a」關係，空心三角形指向父類別。
7. 為了保留 [Polymorphism](#)，關聯應盡量連到抽象父類別或 interface。
8. [Realization](#) 表示 class 實作 interface，是跨平台與抽象化設計的重要基礎。
9. [Aggregation](#) 與 [Composition](#) 都是 whole-part 關係，但 composition 控制 part 的生命週期。
10. [Object Diagram](#) 是某一瞬間的 object snapshot，不是 class-level 繼承圖。
11. Class-scope attribute / operation 等同於 [static](#)。
12. [Dynamic Classification](#) 不能直接靠繼承實作，通常要用 [State Design Pattern](#)。
13. Derived attributes / associations 用 / 標示，代表可以從其他資料推導出來。
14. Qualified association 類似 map / dictionary，用 key 限定關聯對象。
15. Association class 用來表示「關聯本身也有屬性或行為」。
16. Parameterized class 對應 C++ template 或 Java generics。
17. Classification 與 Generalization 不同，繼承設計不能只依賴自然語言的「是一種」。

## 🔗 Related Notes

- [📖 School/OOAD\\_Object-Oriented Analysis and Design](#)
- [School/OOAD/CH11\\_Refactoring\\_and\\_State\\_Pattern](#)
- [School/OOAD/CH10\\_Model-Based\\_Software\\_Engineering\\_and\\_UML](#)

- [UML](#)
- [State Design Pattern](#)